



Brian Simonson

PRINCIPAL ENGINEER

Shamrock Trading Corporation is a world-leading provider of transportation, financial and technology services, helping businesses move freight, improve cash flow and streamline operations.



4

BRANDS



12

LOCATIONS



40

LANGUAGES
SPOKEN



40

COUNTRIES
SERVED



2.5k

EMPLOYEES

◆ TEMPORAL · ENGINEERING TALK

A LIVE-CODED CASE STUDY

Lifting the Veil

How to match your use case & Temporal — one prompt, one AI coding agent, and one insurance claim.

1

prompt → full app

0

databases used

AGENDA

Where we're headed

- 01 The prompt experiment — before & after
- 02 What actually got built (architecture + lifecycle)
- 03 Live demo — real failures, real recovery
- 04 Lessons — how to match your use case to Temporal
- 05 Resources & how to try this yourself

THE QUESTION EVERYONE ACTUALLY HAS

**"This looks powerful. But
is it right for *my* use
case?"**

— every engineer's first honest reaction to Temporal

THE EXPERIMENT

I asked AI a vague prompt. Here's what happened.

Not "here is a finished spec, build it." A genuinely rough, one-paragraph idea.

STEP 1

Write the rough idea

STEP 2

Ask AI to turn it into a real spec

STEP 3

Hand that spec to a coding agent & build it

01

The Prompt Experiment

A concrete way to see what Temporal can do for your use case.

WHAT I ACTUALLY TYPED

My first prompt (verbatim)

● ● ● chat – untitled idea

```
> I want to see what temporal workflows can do for me. Create me a project with a UI to track workflow progress and for me to interact with the workflows. Scenario: Claim process for an insurance company. No database, just get state and progress from temporal itself. Use a local temporal instance. Implement any cool ideas you can think of. Make sure there is at lease 1 wait for signal from the UI. Use local storage so the UI persists on refresh.
```

WHAT IT BECAME

An excerpt from the full implementation prompt

● ● ● prompt – Temporal claims demo spec

```
> Create a working monorepo using:  
- Temporal TypeScript SDK  
- A local Temporal development server  
- NestJS for the backend/API and Temporal worker  
- React with TypeScript for the UI  
- Docker Compose where useful
```

Do not use a database. Temporal must be the source of truth for workflow state and progress.

The human-review step should wait indefinitely for an adjuster to approve, deny, or request more information. The UI must clearly show that the workflow is waiting for input and provide controls for sending the appropriate signal.

READ IT AGAIN, LIKE A CODING AGENT HAS TO

What is not said

"...track workflow progress... interact with the workflows... no database... any cool ideas... at lease 1 wait for signal... local storage so it persists..."

None of this makes the prompt bad.

- ? Which failures are transient vs. permanent?
- ? What must never be lost on a crash?
- ? Where's the API/worker/UI boundary?
- ? What does "interact with" mean — signals? updates?
- ? "Cool ideas" = infinite scope, zero acceptance criteria

THE MOVE THAT ACTUALLY MATTERS

I didn't fix the prompt myself. I asked AI to fix it.

WHY NOT WRITE THE SPEC BY HAND?

Because "what should a good Temporal spec contain" is itself a question worth delegating — the model already knows the shape of retries, signals, queries, sagas, especially with a Temporal skill loaded for grounding.

WHAT I ASKED FOR

"Would this be good for a coding-agent prompt?" — the answer covered architecture, failure modes, and UI/UX explicitly.

Temporal AI skill: github.com/temporalio/skill-temporal-developer

WHAT CAME BACK

The improved prompt

condensed

STACK & ARCHITECTURE, NAMED EXPLICITLY

Temporal TS SDK · local dev server · NestJS API + worker · React + TS UI
· Docker Compose · separate workflow / activity / client / API / UI layers

DURABILITY REQUIREMENTS, NAMED EXPLICITLY

retry policies per activity · at least one timer · a child workflow · signals
for every human action · queries for state/progress · cancellation
handling · failure recovery path

THE CLAIM LIFECYCLE, SPELLED OUT

9 named stages from submission to payout — not "any cool ideas," a
concrete process to map onto primitives

DEFINITION OF DONE

README, unit tests, one integration test that signals a real workflow to
completion, build/lint/test must pass before calling it finished

THE POINT OF THIS WHOLE TALK

**Don't be overwhelmed.
You don't need to spell
everything out to see
what Temporal can do for
you.**

A sharper prompt makes for a sharper demo — but retries, signals, queries and durability are primitives Temporal gives you either way.

02

What Got Built

An insurance claims console where Temporal — not a database — is the source of truth.

ARCHITECTURE

Four pieces, one source of truth

REACT DASHBOARD

Polls the API and keeps only preferences + recent workflow IDs in localStorage.

NESTJS API

REST facade that starts workflows, sends signals, and runs workflow queries.

TEMPORAL SERVER

Event history, task queues, timers, visibility, and the authoritative claim state.

NESTJS WORKER

Runs deterministic workflow code and side-effecting activity implementations.

✗ no database, anywhere

✓ Temporal event history is the state

✓ localStorage = prefs + recent ids only

THE BUSINESS PROCESS

Nine stages

1. Submitted

workflow start

2. Validation

activity + retry

3. Coverage check

activity + retry

4. Damage assessment

heartbeat + cancel

5. Fraud evaluation

child workflow

6. Human review

signal · waits forever

7. Decision

saga: release/reserve

8. Payment

durable timer + activity

9. Completed

workflow result

Every chip on this rail is a query result, live, from a running Temporal workflow — not a hand-maintained progress bar.

Inside the three packages

PACKAGES/WEB — DASHBOARD

```
src/  
├── pages/  
│   ├── DashboardPage.tsx  
│   └── ClaimDetailPage.tsx  
├── components/  
│   ├── StageTimeline.tsx  
│   ├── ActionPanel.tsx  
│   ├── SimulationPanel.tsx  
│   └── ...10 more  
├── hooks/  
│   ├── usePolling.ts  
│   └── usePreferences.tsx  
└── lib/  
    ├── api.ts  
    └── storage.ts
```

PACKAGES/API — REST FACADE

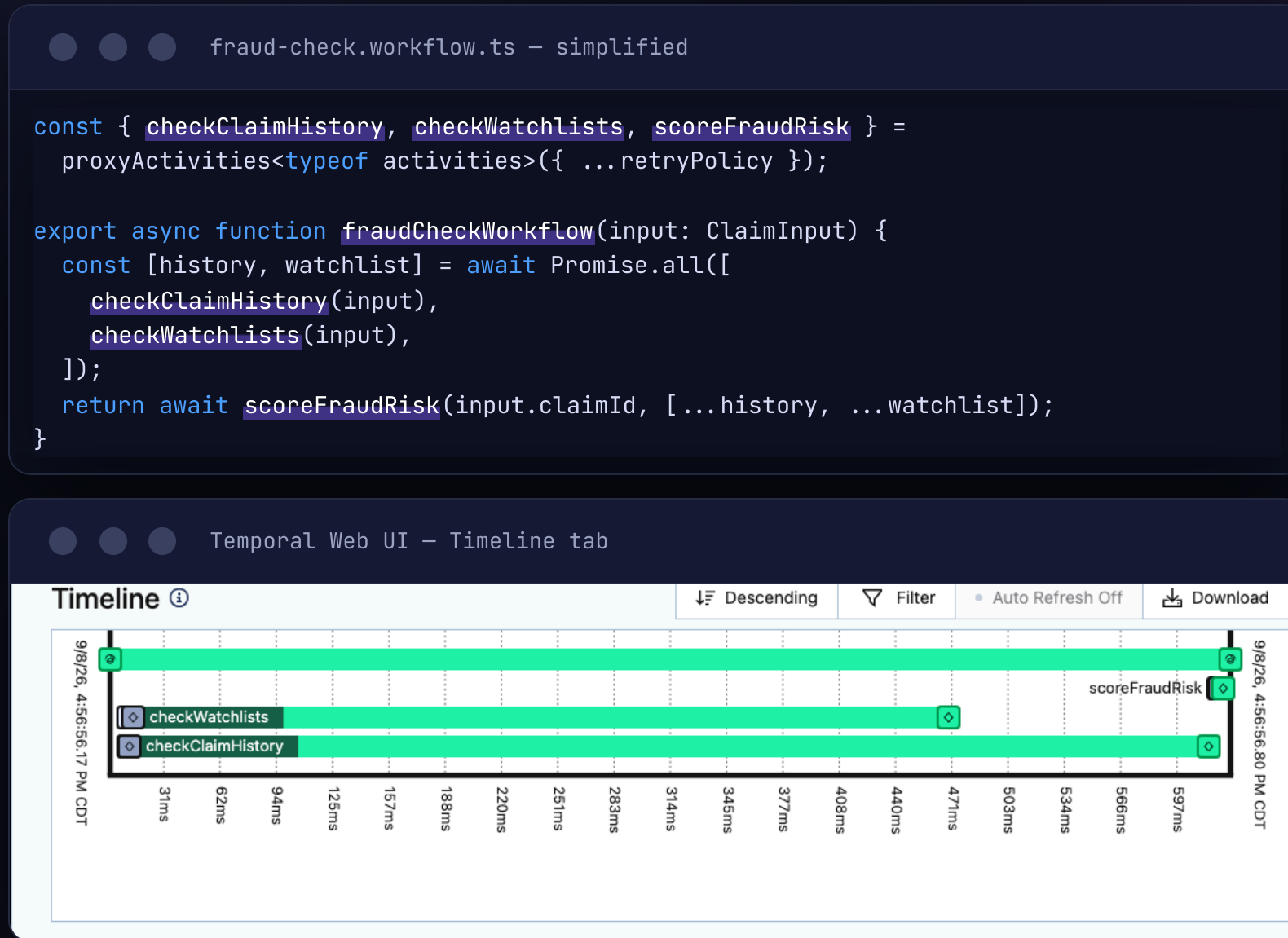
```
src/  
├── claims/  
│   ├── claims.controller.ts  
│   ├── claims.service.ts  
│   ├── temporal.mappers.ts  
│   └── dto.ts  
├── temporal/  
│   └── temporal.service.ts  
├── system/  
│   └── system.controller.ts  
└── main.ts
```

PACKAGES/WORKER — WORKFLOWS & ACTIVITIES

```
src/  
├── workflows/  
│   ├── claim.workflow.ts  
│   └── fraud-check.workflow.ts  
├── activities/  
│   ├── claim/  
│   │   ├── validate-claim.ts  
│   │   ├── process-payment.ts  
│   │   └── ...8 more  
│   ├── fraud/  
│   │   ├── check-watchlists.ts  
│   │   └── score-fraud-risk.ts  
│   └── simulation.ts  
├── temporal/  
│   └── temporal-worker.service.ts  
└── admin/  
    └── admin.controller.ts
```

Plus `packages/shared` — request/response types, stage definitions and progress math every package imports, none of them redefine.

Three activity calls. One real execution timeline.



No custom logging, no tracing setup, no dashboard to build — every activity call at the top shows up as a bar at the bottom the moment it runs.

What's actually demonstrated

RETRY POLICIES

per-stage, configurable backoff

NON-RETRYABLE FAILURES

permanent errors stop cleanly

DURABLE TIMERS

settlement hold + reminder loop

CHILD WORKFLOW

fraud check, own history

SIGNALS

approve / deny / info / recover

QUERIES

live state + progress %

UPDATE + VALIDATOR

adjuster notes, rejected inline

CANCELLATION

saga-style compensation

SEARCH ATTRIBUTES

find claims by status/type/score

WORKER-RESTART DURABILITY

kill it mid-claim, resume

HEARTBEATS

cancellable long-running activity

NO DATABASE

Temporal is the only state store

03

Let's See It Run

Real failures. Real recovery.

DEMO 1 / 6

Submit a claim, watch it run

DEMO 2 / 6

The workflow waits — forever — for you

DEMO 3 / 6

Break it a little — watch it heal itself

DEMO 4 / 6

Break it for real — recover it from the UI

DEMO 5 / 6

Kill the worker. Mid-claim. On purpose.

DEMO 6 / 6

Cancel mid-flight — compensations run

LIVE

Let's Look at the Live Demo and Temporal UI

04

Matching Your Use Case

PATTERN MATCH

Signals you have a good Temporal use case

- ✓ A process that spans minutes, days, or months — not one request/response
- ✓ Short-lived workflows can still fit — milliseconds or seconds are fine when you need durability, retries, or history
- ✓ A human has to approve, deny, or provide info at some unknown point
- ✓ Steps call flaky external systems that need retries with backoff
- ✓ A crash mid-process must not lose or double-run anything
- ✓ You need an audit trail of exactly what happened, in order
- ✓ Multiple steps must be undone together if a later one fails (saga)

THE REUSABLE CHECKLIST

Durability questions to ask yourself

ASK ABOUT FAILURE

What activities can fail transiently? What's permanent? What must be non-retryable?

ASK ABOUT WAITING

What waits on a human? For how long? What happens if nobody answers?

ASK ABOUT STATE

What must survive a crash? Who is the source of truth — your DB, or the workflow history?

ASK ABOUT BOUNDARIES

Where does UI end and API begin? Where does API end and worker begin?

Answer these four categories before you design the system, and your Temporal model gets much easier to see.

HOMEWORK

Try this yourself, this week

● ● ● prompt-template.txt

```
> Here's my rough idea: [describe your process in 2-3 sentences]. Turn this into a spec for a coding agent building it on Temporal.  
Explicitly call out: which steps need retries and what kind of failures they should tolerate, where a human needs to approve/reject/provide  
input and how long we should wait, what a crash mid-process must not lose, and how the UI/API/worker boundaries should be drawn. Then build  
it.
```

GO BUILD SOMETHING

Resources

THIS DEMO

Full source, README, tests — github.com/simy307/temporal-claims-demo

TEMPORAL DOCS

docs.temporal.io · TypeScript SDK samples

COMMUNITY

Temporal Slack · community forum · local meetups

THE PROMPT TEMPLATE

Previous slide — steal it, adapt it, use it on your own use case

 TEMPORAL · ENGINEERING TALK

Thank you.